PRÁCTICA 1

Algoritmia y Estructuras de Datos Avanzadas

Medición de tiempos, RLE y componentes conexas

Memoria

Curso 2026–2027

Alejandro Dominguez e Iker Romero

1. Objetivo

El objetivo de esta práctica es estudiar el comportamiento de varios algoritmos, contrastar su complejidad teórica con tiempos medidos y justificar el funcionamiento del TAD Conjunto Disjunto aplicado a la búsqueda de componentes conexas de un grafo no dirigido.

La memoria se centra en los experimentos solicitados: *has_sum_pair* en casos favorable y desfavorable, y la comparación de *rle_encode_naive* con *rle_encode_optimized* usando entradas con pocas y muchas rachas. Finalmente se razona la corrección y el coste de *ccs*.

2. Metodología experimental

Se ha utilizado la implementación entregada en *p1.py*. Para cada tamaño de entrada se ha preparado la lista antes de iniciar el cronómetro, de forma que el tiempo medido corresponde a la ejecución del algoritmo y no a la generación de datos. Se ha empleado *time_measure()*, adecuado para medir intervalos cortos.

En cada medición parcial se ejecuta la función Nrep veces sobre una misma entrada y se divide el tiempo total entre Nrep. Este proceso se repite Nstat veces con entradas preparadas por separado. Se calcula la media aritmética de los tiempos parciales y la varianza poblacional.

has_sum_pair: Nlist = [100, 500, 1000, 2000, 4000, 8000], Nrep = 100 y Nstat = 20. **RLE:** Nlist = [100, 250, 500, 1000, 2000], Nrep = 3 y Nstat = 8. Se han usado menos repeticiones para RLE porque la versión naive puede ser costosa cuando la entrada contiene muchas rachas.

3. Experimento con has_sum_pair

3.1. Descripción del algoritmo

La función recorre la lista y mantiene un conjunto llamado *vistos*. Para cada número x calcula el complemento target − x. Si ese complemento ya está en el conjunto, devuelve True; en caso contrario, añade x a *vistos*. Si termina el recorrido sin encontrar pareja, devuelve False. El uso de un conjunto permite búsquedas de pertenencia en tiempo esperado O(1), de modo que el coste esperado del recorrido completo es O(n), con espacio adicional O(n).

La condición de índices distintos se cumple porque se comprueba si el complemento está en *vistos* antes de añadir el elemento actual. Por tanto, un único 5 no puede formar una pareja que sume 10 consigo mismo, mientras que dos valores 5 en posiciones distintas sí.

3.2. Preparación de los casos

Caso favorable. La lista comienza por [1, 9] y se completa con ceros hasta alcanzar el tamaño n. Con target = 10, la pareja se encuentra al procesar el segundo elemento, así que el algoritmo termina prácticamente al principio.

Caso desfavorable. La lista contiene únicamente ceros y se usa target = 1. No existe ninguna pareja válida, por lo que la función debe examinar los n elementos antes de devolver False.

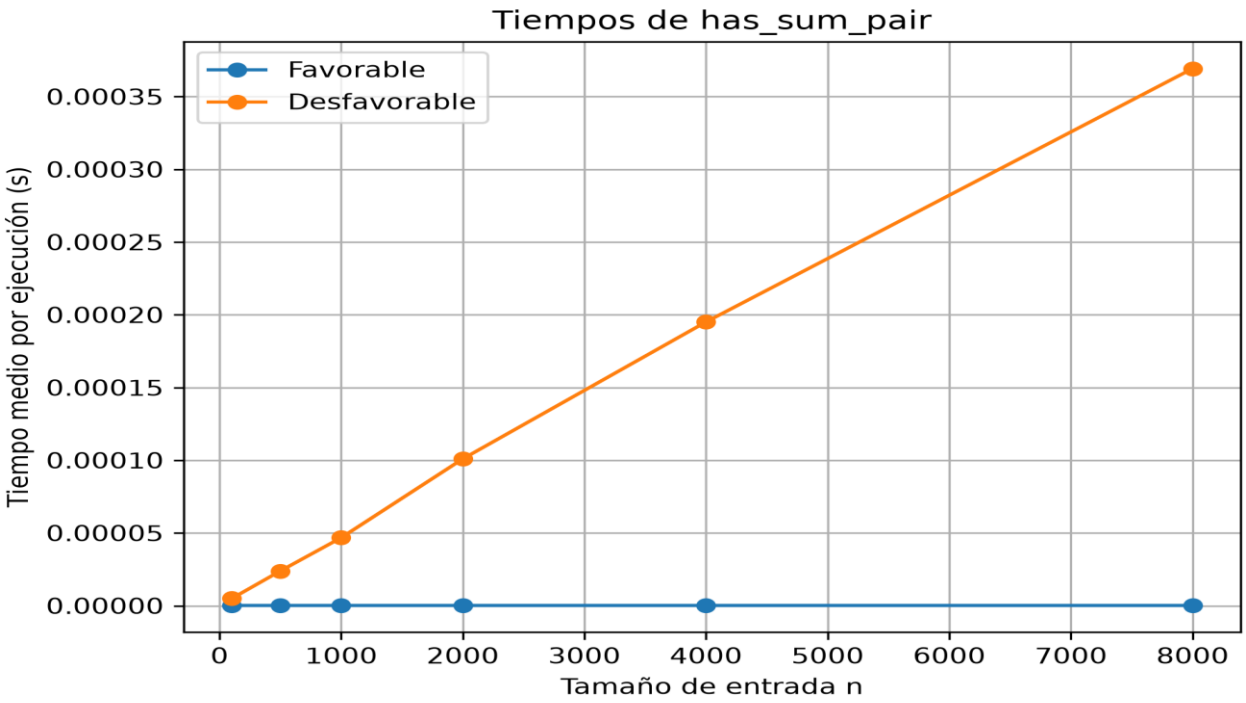


Figura 1. Tiempo medio por ejecución en los casos favorable y desfavorable.

3.3. Resultados de has_sum_pair

n	Favorable (µs)	Desfavorable (µs)
100	0.362	6.841
500	0.227	25.108
1000	0.267	49.559
2000	0.218	97.444
4000	0.229	194.034
8000	0.212	379.277

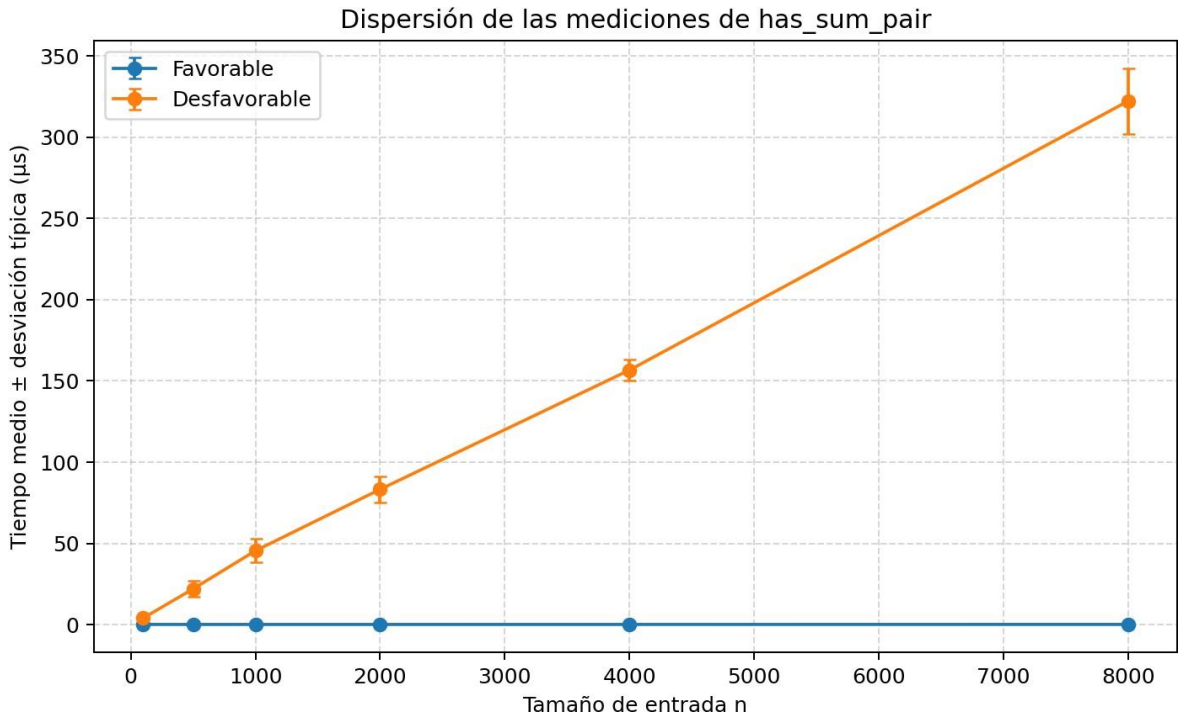


Figura 2. Media y dispersión aproximada representada mediante una desviación típica de las mediciones parciales.

3.4. Interpretación

En el caso favorable, el tiempo permanece aproximadamente constante al aumentar n: se observa alrededor de 0,21–0,36 microsegundos por llamada en esta ejecución. Esto se explica porque la función encuentra la pareja al principio y no necesita recorrer el resto de la lista. Para este patrón concreto, el trabajo realizado es O(1) respecto del tamaño total de la entrada.

En el caso desfavorable, el tiempo crece al aumentar n: pasa de unos 6.8 µs para n = 100 a unos 379 µs para n = 8000. Como no hay pareja, se recorre toda la lista. La tendencia es coherente con un coste esperado O(n). Las pequeñas irregularidades que puedan aparecer en mediciones individuales se deben a ruido de ejecución, planificación del sistema operativo y otros factores externos.

La varianza permite estudiar la estabilidad de las mediciones: una varianza mayor indica más dispersión entre las mediciones parciales. En la figura se muestra la desviación típica, raíz cuadrada de la varianza, porque está expresada en las mismas unidades que el tiempo. Las barras pequeñas pueden quedar visualmente ocultas por la escala de la gráfica.

4. Comparación de las implementaciones RLE

4.1. Qué comprime RLE

Run-Length Encoding (RLE) representa cada racha de elementos consecutivos iguales mediante una pareja (elemento, longitud). Por ejemplo, ['A','A','A','B','B','C','A'] se codifica como [('A',3), ('B',2), ('C',1), ('A',1)]. La última A forma una nueva racha porque no es consecutiva a las primeras A.

Las dos implementaciones recorren la lista, cuentan los elementos iguales consecutivos y guardan cada racha. La diferencia principal está en cómo se añade cada pareja a la lista de salida.

4.2. Diferencia de coste

La versión optimizada utiliza *append*, que añade cada pareja al final de la lista de salida. El recorrido y la construcción tienen coste esperado O(n). La versión naive construye una lista nueva en cada concatenación *output_lst = output_lst + [(elemento, contador)]*. Cada concatenación copia la salida acumulada; si hay muchas rachas, el coste acumulado puede ser $O(n^2)$.

Para distinguir ambos comportamientos se han usado dos patrones. En el caso de pocas rachas, todos los elementos son 'A' y existe una única racha. En el caso de muchas rachas, los elementos alternan entre 'A' y 'B', de modo que casi cada posición genera una racha nueva.

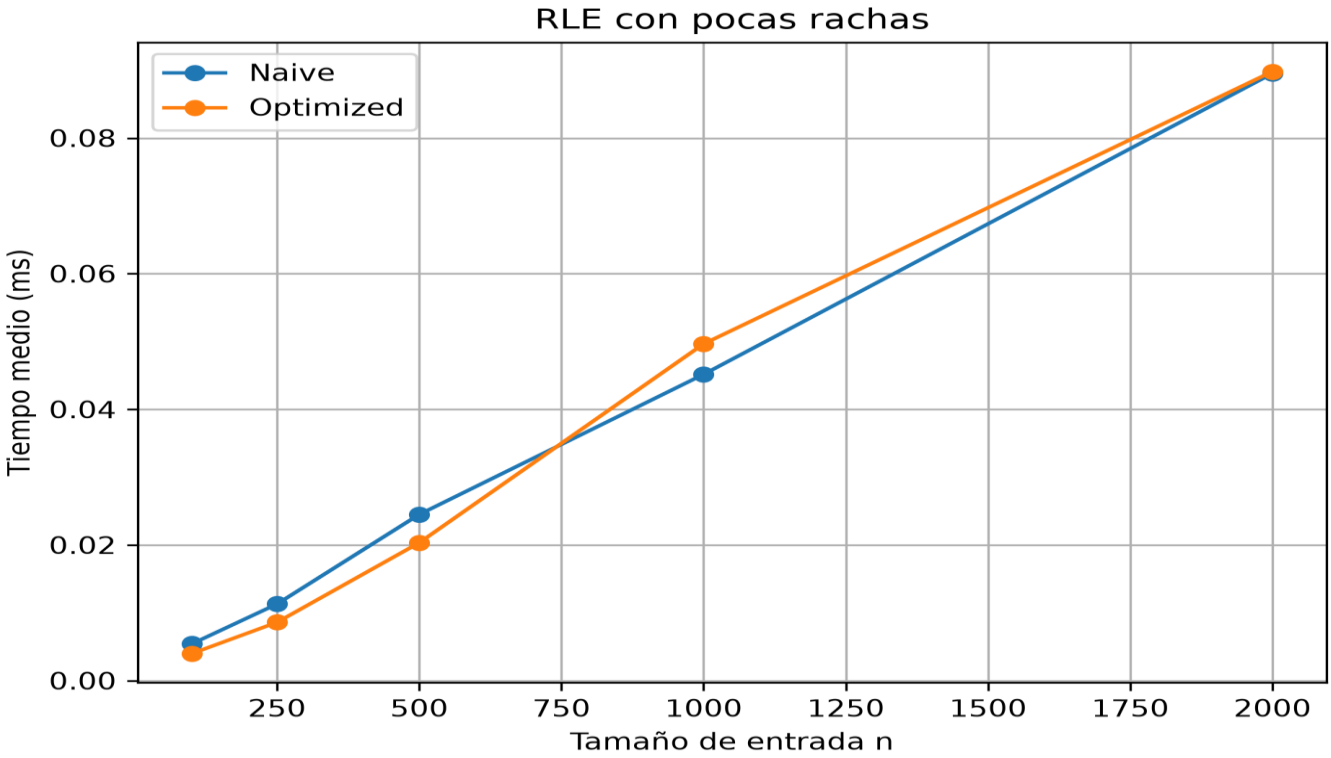


Figura 3. Comparación RLE con una sola racha.

4.3. Resultados con muchas rachas

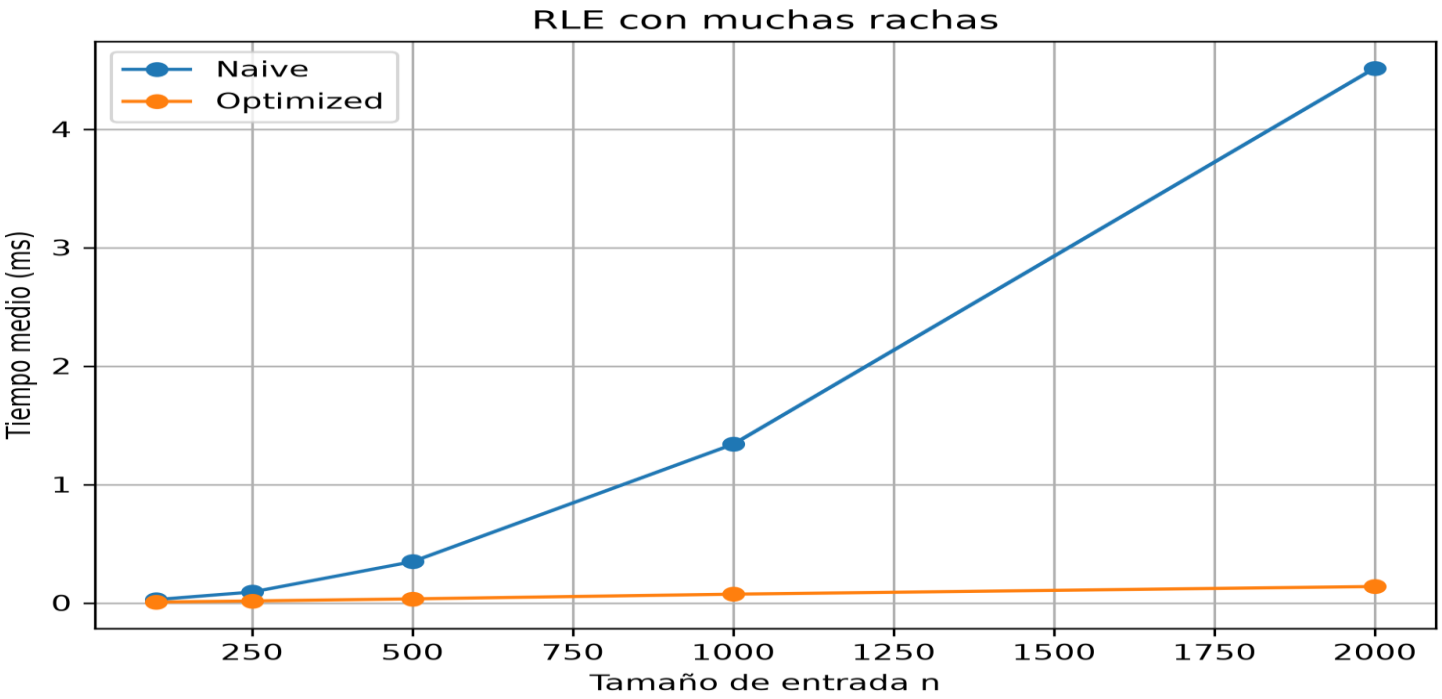


Figura 4. Comparación RLE con elementos alternos y muchas rachas.

4.4. Tabla de resultados

n	Naive: pocas (ms)	Optimizada: pocas (ms)	Naive: muchas (ms)	Optimizada: muchas (ms)
100	0.0056	0.0070	0.0441	0.0132
250	0.01	0.0161	0.1719	0.0327
500	0.0252	0.0340	0.5609	0.0665
1000	0.0782	0.0808	2.0948	0.0950
2000	0.1695	0.1565	7.2939	0.2092

4.5. Interpretación

Con pocas rachas, ambas versiones muestran tiempos parecidos. En este patrón solo se añade una pareja a la salida, así que la concatenación no copia repetidamente una lista de salida grande. El coste observado es compatible con un recorrido lineal, y la diferencia entre las versiones es pequeña.

Con muchas rachas, las diferencias son claras. Para n = 2000, la versión naive tarda aproximadamente 7,29 ms, frente a 0,21 ms de la optimizada en esta ejecución. La versión naive se ralentiza porque concatena repetidame listas cada vez mayores; la versión optimizada añade cada racha mediante append y mantiene un crecimiento mucho más cercano a O(n).

Los tiempos medidos no tienen por qué duplicarse exactamente cuando se duplica n: la complejidad describe una tendencia asintótica, mientras que las medidas concretas dependen de la implementación y del entorno. Aun así, el patrón de muchas rachas permite observar el efecto de las copias repetidas con más claridad.

5. Corrección y coste de ccs

La función ccs(n, l) inicializa un Conjunto Disjunto para los vértices 0, …, n−1. Para cada rama (u, v), calcula los representantes de ambos extremos y, si son distintos, los une. Finalmente convierte la estructura en un diccionario mediante cd_2_dict. Cada clave representa un conjunto y su lista contiene los vértices de esa componente.

5.1 Sin darnos cuenta, en nuestro algoritmo de encontrar CCs podemos pasar listas con ramas repetidas o donde los vértices coinciden con los de una que ya está, aunque en orden inverso. ¿Afectará esto al resultado del algoritmo? ¿Por qué?

No afectará al resultado final, ya que al procesar una arista y unir sus nodos en un subconjunto disjunto el representante de ambas se vuelve el mismo, es decir, si tenemos una arista (1,2) al principio estará dividida en dos subconjuntos {1} y {2}, tras hacer el union(1,2), uno de los dos nodos cuelga del otro, lo que hace que su representante sea el mismo. Lo que si provocará es que la función realice operaciones innecesarias, procesando la misma arista varias veces.

5.2 Argumentar que nuestro algoritmo de encontrar componentes conexas es correcto, esto es, que su final en los distintos subconjuntos disjuntos se encuentran los vértices de las distintas componentes del grafo dado.

Nuestro algoritmo se desarrolla de forma correcta debido a varias razones. En primer lugar, al inicializar todos los nodos como conjuntos disjuntos cada uno, nos aseguramos de que si hay algún nodo que no tenga conexión con otro nodo, al desarrollarlo en forma de diccionario se muestre como un nodo aislado. Además, el bucle for nos asegura que todas las aristas sean procesadas con éxito. Finalmente, las operaciones find y union procesan cada arista, provocando que si dos vértices no comparten la misma raíz, se unen; gracias a esto, cualquier camino de vértices conectados acaba agrupado bajo un mismo representante, garantizando que los subconjuntos finales representen exactamente a las componentes conexas del grafo.

5.3 El tamaño de un grafo no dirigido viene determinado por el número n de nodos y la longitud de la lista l de ramas. Estimar razonadamente en función de ambos el coste del algoritmo de encontrar las componentes conexas mediante conjuntos disjuntos.

El coste de la función init_cd() es de O(n) ya que lo que hace es desarrollar tantos subconjuntos como nodos (n) le hayamos pasado. Como hemos utilizado que el árbol de menor tamaño se enlace siempre al de mayor tamaño, tenemos que la función unión tiene de coste O(log n) y se desarrolla para cada par de aristas l, l x O(log n). Finalmente, convertir la estructura a un diccionario requiere iterar sobre los n vértices realizando una última operación find por cada uno, lo que suma un coste de O(n log n). Por tanto, el coste total estimado es la suma de los procesos.

6. Conclusiones

Los experimentos permiten distinguir el coste teórico de los algoritmos según el patrón de entrada. En *has_sum_pair*, una pareja al principio produce un caso favorable prácticamente constante, mientras que la ausencia de pareja obliga a recorrer toda la lista y genera un crecimiento esperado lineal.

La comparación RLE muestra que el patrón de entrada importa. Con una sola racha, las dos versiones presentan tiempos similares; con muchas rachas, la concatenación sucesiva de listas de la versión naive produce un coste mucho mayor. El uso de append en la versión optimizada evita las copias acumuladas y ofrece un crecimiento esperado lineal.

Por último, ccs calcula las componentes conexas usando Conjunto Disjunto: cada rama une conjuntos que deben estar conectados, las ramas repetidas o invertidas no cambian la partición y los vértices aislados se conservan gracias a la inicialización de todos los vértices, consiguiendo una complejidad casi lineal.